



AWS CloudFormation

© 2020, Amazon Web Services, Inc. ou suas afiliadas. Todos os direitos reservados.

Este módulo descreve o serviço AWS CloudFormation.

O que você aprenderá

Principais pontos da lição

Você aprenderá a:

- Descrever a finalidade do AWS CloudFormation.
- Criar um modelo do AWS CloudFormation.
- Usar as práticas recomendadas do AWS CloudFormation.

Principais termos:

- Modelo do AWS CloudFormation
- Pilha do AWS CloudFormation



Neste módulo, você aprenderá a:

- Descrever a finalidade do AWS CloudFormation.
- Criar um modelo do AWS CloudFormation.
- Usar as práticas recomendadas do AWS CloudFormation.

O desafio da implantação na nuvem

Apresentando o AWS Lambda.

O AWS Lambda é um serviço de computação sem servidor orientado por eventos. Você pode usar o Lambda para executar código sem provisionar ou gerenciar servidores. Você paga apenas pelo tempo de computação consumido. Não haverá cobranças quando seu código não estiver em execução.

Desafios de implantação na nuvem

Os desafios de implantação da nuvem incluem:

Implantar lançamentos em vários locais geográficos

Depurar implantações

Documentar todas as alterações

Atualização de servidores ativos

Ter a capacidade de gerenciar uma reversão

Gerenciar dependências em sistemas e subsistemas

Implantar ambientes repetíveis e idênticos

4



A nuvem abre muitas possibilidades. No entanto, ela também coloca dúvidas sobre como gerenciar o seu poder e flexibilidade, tais como:

- Como atualizar servidores que já foram implantados em um ambiente de produção?
- Como você implanta consistentemente uma infraestrutura em várias regiões em localizações geográficas diferentes?

A nuvem abre muitas possibilidades. No entanto, ela também coloca dúvidas sobre como gerenciar o seu poder e flexibilidade, tais como:

- Como atualizar servidores que já foram implantados em um ambiente de produção?
- Como você implanta consistentemente uma infraestrutura em várias regiões em localizações geográficas diferentes?
- Como reverter uma implantação que não foi executada de acordo com o plano? Em outras palavras, como você recupera os recursos que já foram criados?
- Como testar e depurar uma implantação antes de implantá-la em produção?
- Como você gerencia dependências de sistemas e tecnologias e também de subsistemas inteiros (por exemplo, um site implantado sobre uma infraestrutura de comércio eletrônico)?



AWS CloudFormation

Apresentando o AWS Lambda.

O AWS Lambda é um serviço de computação sem servidor orientado por eventos. Você pode usar o Lambda para executar código sem provisionar ou gerenciar servidores. Você paga apenas pelo tempo de computação consumido. Não haverá cobranças quando seu código não estiver em execução.

AWS CloudFormation

- Modela e provisiona recursos de infraestrutura de nuvem.
- Oferece suporte à maioria dos serviços da AWS.
- Cria, atualiza e exclui um conjunto de recursos como uma unidade única chamada *pilha*.
- Detecta alterações, chamadas de "desvio", na pilha e em recursos individuais.



O AWS CloudFormation permite criar e provisionar implantações de infraestrutura da AWS de forma previsível e repetida. Você pode usar o AWS CloudFormation em serviços como Amazon Elastic Compute Cloud (Amazon EC2), Amazon Elastic Block Store (Amazon EBS), Amazon Simple Notification Service (Amazon SNS), Elastic Load Balancing e Auto Scaling. O AWS CloudFormation permite que você use um arquivo de modelo para criar e excluir um conjunto de recursos, que são gerenciados juntos como uma única unidade (uma *pilha*).

Embora você gerencie os seus recursos por meio do AWS CloudFormation, os

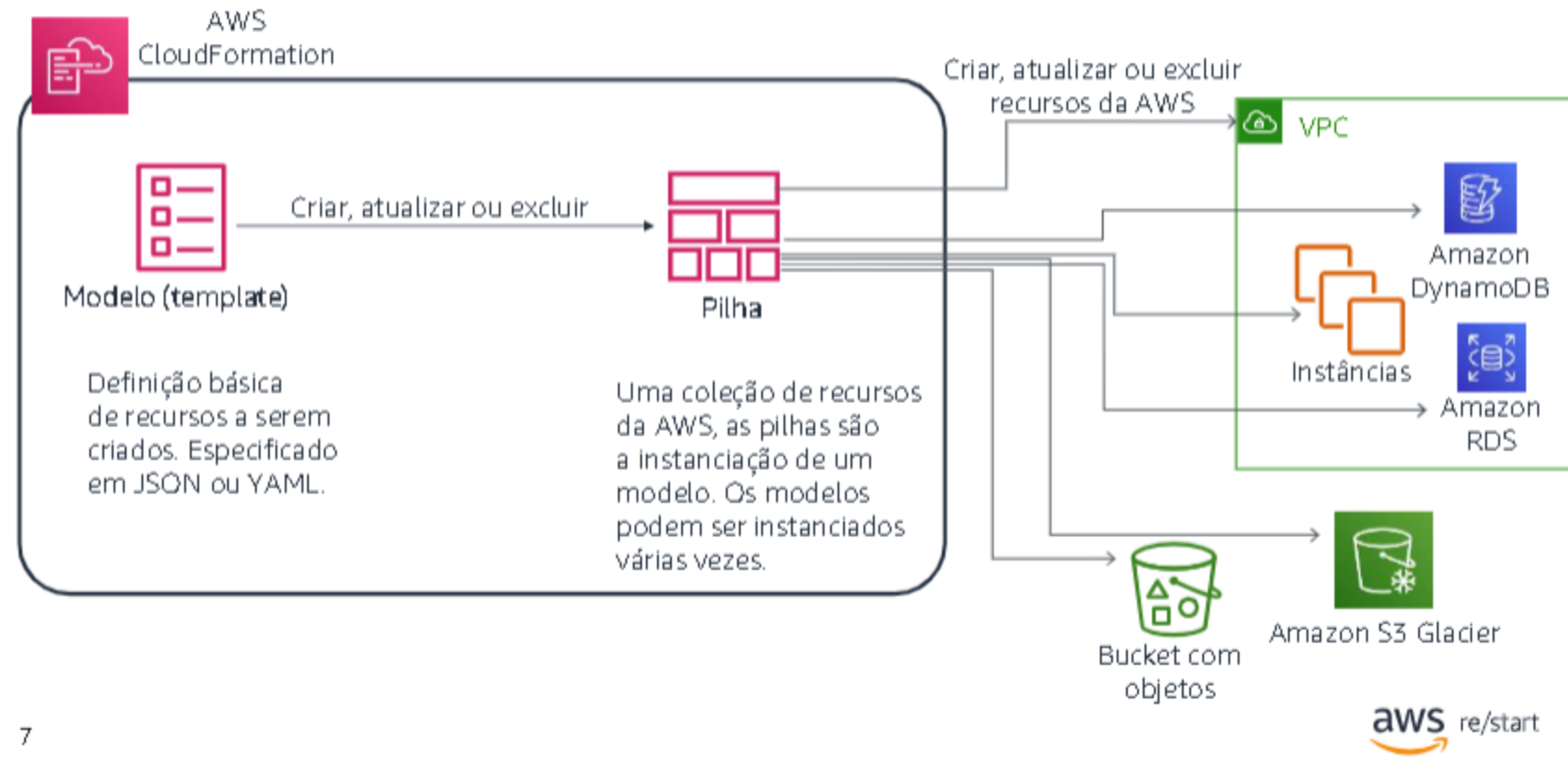
O AWS CloudFormation permite criar e provisionar implantações de infraestrutura da AWS de forma previsível e repetida. Você pode usar o AWS CloudFormation em serviços como Amazon Elastic Compute Cloud (Amazon EC2), Amazon Elastic Block Store (Amazon EBS), Amazon Simple Notification Service (Amazon SNS), Elastic Load Balancing e Auto Scaling. O AWS CloudFormation permite que você use um arquivo de modelo para criar e excluir um conjunto de recursos, que são gerenciados juntos como uma única unidade (uma *pilha*).

Embora você gerencie os seus recursos por meio do AWS CloudFormation, os usuários podem alterar esses recursos *fora* do AWS CloudFormation. Os usuários podem editar recursos diretamente usando o serviço subjacente que criou o recurso. Por exemplo, você pode usar o console do Amazon EC2 para atualizar uma instância de servidor que foi criada como parte de uma pilha do AWS CloudFormation. Algumas alterações podem ser acidentais e outras podem ser feitas intencionalmente para responder a eventos operacionais urgentes. Independentemente disso, as alterações feitas fora do AWS CloudFormation podem complicar as operações de atualização ou exclusão da pilha.

Você pode usar a *detecção de desvio* para identificar quais recursos de pilha têm alterações de configuração que foram feitas fora do gerenciamento do AWS

CloudFormation. Em seguida, você pode executar uma ação corretiva para que os seus recursos de pilha estejam novamente sincronizados com suas definições no modelo de pilha. Por exemplo, você pode atualizar os recursos desviados diretamente para que eles concordem com sua definição de modelo. Resolver o desvio ajuda a garantir a consistência de sua configuração e o sucesso de suas operações de pilha.

Terminologia do AWS CloudFormation



Dois termos principais para o AWS CloudFormation são *modelos* e *pilhas*.

Um *modelo* é uma especificação dos recursos da AWS a serem provisionados.

Uma *pilha* é uma coleção de recursos da AWS que foram criados a partir de um modelo. Você pode provisionar (criar) uma pilha várias vezes.

Quando uma pilha é *provisionada*, os recursos da AWS especificados pelo modelo de pilha são criados. Todas as cobranças incorridas pelo uso desses serviços começarão a ser geradas quando forem criadas como parte da pilha do AWS CloudFormation.

Dois termos principais para o AWS CloudFormation são *modelos* e *pilhas*.

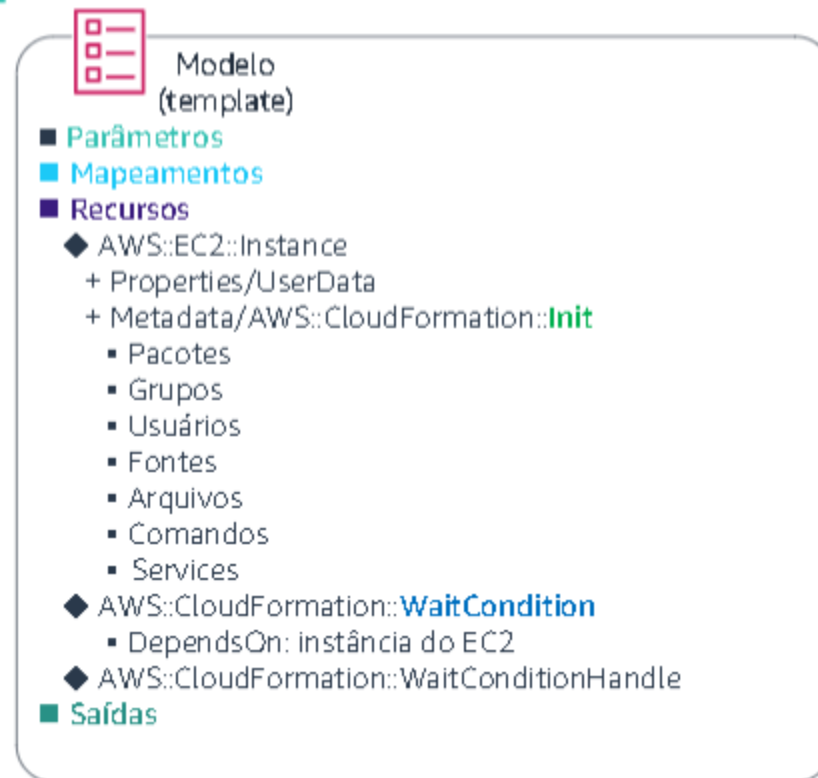
Um *modelo* é uma especificação dos recursos da AWS a serem provisionados.

Uma *pilha* é uma coleção de recursos da AWS que foram criados a partir de um modelo. Você pode provisionar (criar) uma pilha várias vezes.

Quando uma pilha é *provisionada*, os recursos da AWS especificados pelo modelo de pilha são criados. Todas as cobranças incorridas pelo uso desses serviços começarão a ser geradas quando forem criadas como parte da pilha do AWS CloudFormation.

Quando uma pilha é *excluída*, os recursos associados a essa pilha são excluídos. A ordem de exclusão é determinada pelo AWS CloudFormation. Você não tem controle direto sobre o que e quando é excluído.

Estrutura do modelo



Parâmetros: entradas no modelo

Mapeamentos: variáveis estáticas — geralmente as imagens de máquina da Amazon (AMIs) mais recentes.

Recursos: ativos da AWS para criar.

Init: aplicativos personalizados a serem executados durante a inicialização.

WaitCondition: sinal da instância de que os dados do usuário concluíram a execução.

Saídas: valores de recursos personalizados criados pelo modelo (URLs, nomes de usuário etc.).



Esses elementos são os principais componentes de um modelo do AWS CloudFormation.

Parâmetros é uma seção *opcional* do modelo. Quando os parâmetros estão presentes, eles incluem pares de nome-valor. Esses parâmetros podem ser referenciados posteriormente no modelo.

Os mapeamentos são uma seção opcional do modelo. Normalmente, os mapeamentos são usados para se referir aos valores das imagens de máquina da Amazon (AMIs) mais atuais em

Esses elementos são os principais componentes de um modelo do AWS CloudFormation.

Parâmetros é uma seção *opcional* do modelo. Quando os parâmetros estão presentes, eles incluem pares de nome-valor. Esses parâmetros podem ser referenciados posteriormente no modelo.

Os mapeamentos são uma seção *opcional* do modelo. Normalmente, os mapeamentos são usados para se referir aos valores das imagens de máquina da Amazon (AMIs) mais atuais em uma região, pois diferem por região. Eles também mudam com o tempo à medida que novas AMIs são lançadas.

Recursos é uma seção *obrigatória* do modelo. A seção **Recursos** contém os objetos da AWS que você criará. Você pode criar recursos em um grande número de serviços da AWS, incluindo Amazon EC2, Amazon Simple Storage Service (Amazon S3), Amazon Virtual Private Cloud (Amazon VPC) e muitos outros. Você pode especificar relacionamentos e dependências entre recursos para garantir que os recursos sejam criados na ordem correta.

Init. O tipo de recurso CloudFormation::Init permite implantar aplicativos, arquivos e outros recursos nas suas instâncias do EC2 como parte do processo de implantação.

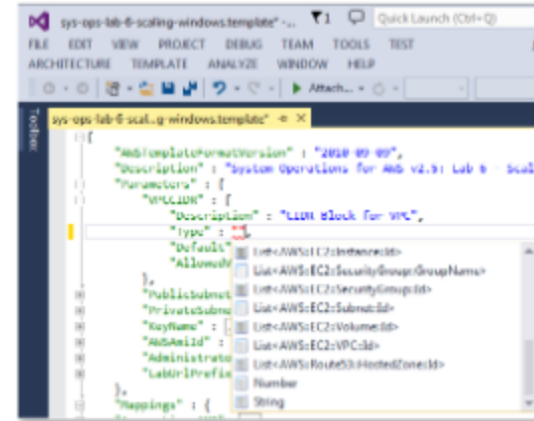
WaitCondition. O tipo de recurso CloudFormation::WaitCondition é usado para coordenar a criação de recursos. Por exemplo, suponha que um recurso tenha uma dependência de outro recurso. O AWS CloudFormation aguardará a conclusão de outras atividades de criação de recursos antes de tentar criar o recurso dependente. O WaitConditions permite que você retorne à AWS quando os comandos do CloudFormation::WaitCondition terminarem de ser executados com êxito.

Saídas são uma seção *opcional* do modelo. A seção **Saídas** retorna valores de string que são criados pelo modelo e que podem ser importantes para os usuários. Por exemplo, se você criar uma instância do Amazon EC2 que funcione como um servidor Web público, poderá optar por produzir o nome DNS (Domain Name Server) público completo do servidor na seção **Saídas**.

Editar modelos do AWS CloudFormation

Use um editor compatível com o AWS CloudFormation — ou um editor JSON ou YAML para aproveitar recursos como **capacidades de análise**, **preenchimento automático** e **verificação de sintaxe**.

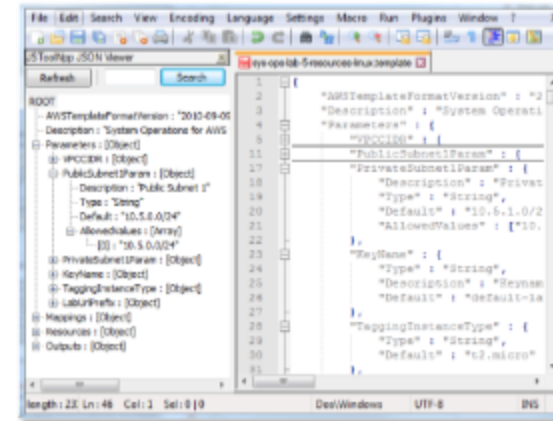
Editor de modelos do AWS CloudFormation para Visual Studio ou Eclipse



Ferramentas de terceiros

```
>>> from troposphere import Ref, Template
>>> import troposphere.ec2 as ec2
>>> t = Template()
>>> instance = ec2.Instance("myinstance")
>>> instance.ImageId = "ami-951945d8"
>>> instance.InstanceType = "t1.micro"
>>> t.add_resource(instance)
<troposphere.ec2.Instance object at 0x101b3390>
>>> print(t.to_json())
{
  "Resources": {
    "myinstance": {
      "Properties": {
        "ImageId": "ami-951945d8",
        "InstanceType": "t1.micro"
      },
      "Type": "AWS::EC2::Instance"
    }
  }
}
```

Editor de texto com plugins JavaScript ou JSON



Embora o formato JSON tenha muitos atributos positivos, uma das desvantagens é que você pode editar o seu caminho em um erro de sintaxe que não se pode resolver facilmente. Se você remover uma chave no início de um documento JSON, poderá passar a próxima hora tentando encontrar a origem do erro. Usar um editor compatível com o AWS CloudFormation ou um editor compatível com JSON simplifica a tarefa de edição de modelos. Da mesma forma, os documentos YAML devem ser cuidadosamente escritos, mantendo os espaços em mente.

Embora o formato JSON tenha muitos atributos positivos, uma das desvantagens é que você pode editar o seu caminho em um erro de sintaxe que não se pode resolver facilmente. Se você remover uma chave no início de um documento JSON, poderá passar a próxima hora tentando encontrar a origem do erro. Usar um editor compatível com o AWS CloudFormation ou um editor compatível com JSON simplifica a tarefa de edição de modelos. Da mesma forma, os documentos YAML devem ser cuidadosamente escritos, mantendo os espaços em mente.

Você pode instalar o AWS Toolkit no Visual Studio ou o Eclipse, que oferece suporte à edição de modelos do AWS CloudFormation. O toolkit adiciona suporte ao AWS CloudFormation a ambos os editores. O suporte inclui análise e formatação, preenchimento automático de comandos e palavras-chave do AWS CloudFormation e verificação de sintaxe. Se preferir trabalhar em um editor de texto (como Sublime Text ou Notepad++), os editores de texto mais populares têm plugins JSON ou YAML que você pode usar para formatar o seu código.

Para obter mais informações sobre como editar modelos do AWS CloudFormation no Visual Studio, consulte [Usando o Editor de modelos do AWS CloudFormation para Visual Studio](#).

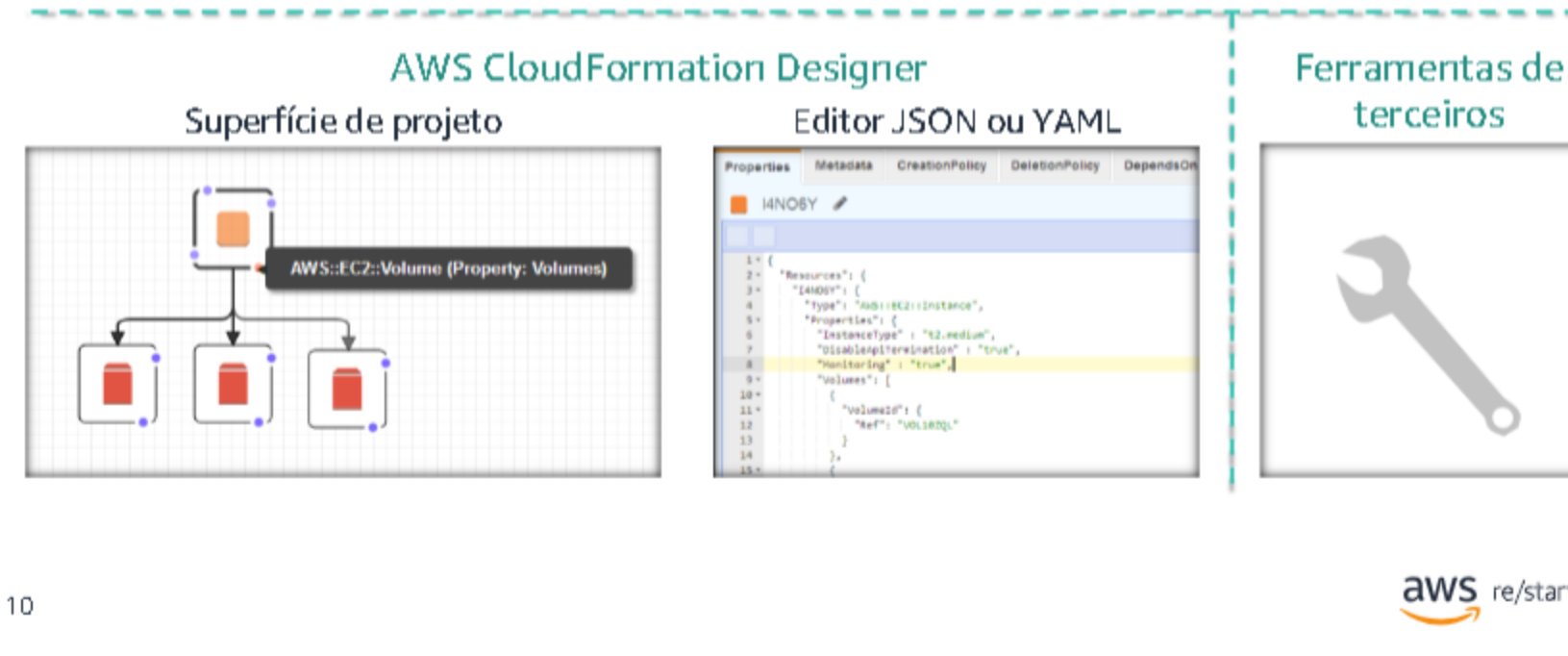
Para obter mais informações sobre como editar modelos do AWS

CloudFormation no Eclipse, consulte [O Editor de modelos do AWS](#)



Modelos de projeto

O **AWS CloudFormation Designer** é uma ferramenta visual que permite criar e modificar modelos do AWS CloudFormation usando uma interface de arrastar e soltar.



10

O AWS CloudFormation Designer é uma ferramenta visual que permite criar e modificar modelos do AWS CloudFormation usando uma interface de arrastar e soltar. Você pode facilmente adicionar, modificar ou remover recursos e o JSON subjacente será alterado de acordo. Se você modificar um modelo que seja associado a uma pilha em execução, você poderá alterar a pilha de modo que ela fique de acordo com o modelo.

Ferramentas de terceiros também podem fornecer soluções para ajudá-lo a projetar modelos do AWS CloudFormation.

Iniciar e excluir pilhas

- Os modelos do AWS CloudFormation podem ser executados como pilhas por meio de:
 - Console de Gerenciamento da AWS;
 - AWS CLI;
 - APIs da AWS.
- Se um erro for encontrado ao iniciar um modelo, todos os recursos serão revertidos por padrão.
- Quando as pilhas são excluídas, os recursos são revertidos.
 - Opcionalmente, você pode habilitar a proteção contra encerramento em uma pilha.

Se um erro for encontrado quando você iniciar um modelo do AWS CloudFormation, todos os recursos serão revertidos por padrão. Você pode alterar essa opção na linha de comando.

O AWS CloudFormation garante que todos os recursos de pilha sejam criados ou excluídos, conforme apropriado. Como o AWS CloudFormation trata os recursos da pilha como uma única unidade, eles devem ser criados ou excluídos com êxito para que a pilha seja criada ou excluída. Se um recurso não puder ser criado, o AWS CloudFormation reverterá a pilha e excluirá automaticamente todos os recursos criados. Se um recurso não pode ser excluído, todos os

Se um erro for encontrado quando você iniciar um modelo do AWS CloudFormation, todos os recursos serão revertidos por padrão. Você pode alterar essa opção na linha de comando.

O AWS CloudFormation garante que todos os recursos de pilha sejam criados ou excluídos, conforme apropriado. Como o AWS CloudFormation trata os recursos da pilha como uma única unidade, eles devem ser criados ou excluídos com êxito para que a pilha seja criada ou excluída. Se um recurso não puder ser criado, o AWS CloudFormation reverterá a pilha e excluirá automaticamente todos os recursos criados. Se um recurso não pode ser excluído, todos os recursos restantes são mantidos até que a pilha possa ser excluída com êxito.

- Para preservar um volume do Amazon Elastic Block Store (Amazon EBS), defina seu atributo *DeleteOnTermination* como **Falso**.
- Para preservar um recurso, defina o atributo *DeletionPolicy* como **Retet**.

Alguns recursos podem não ser excluídos quando uma pilha é excluída, como um bucket do S3.

Opcionalmente, você também pode habilitar a proteção contra encerramento em uma pilha.

Definir parâmetros em um modelo

Para definir parâmetros em um modelo:

- Use a seção **Parâmetros** opcional para personalizar seus modelos.
- Os parâmetros permitem que você insira valores personalizados no seu modelo toda vez que criar ou atualizar uma pilha.

```
"Parameters": {  
  "VPCCIDR": {  
    "Description": "CIDR Block for VPC",  
    "Type": "String",  
    "Default": "10.200.0.0/20",  
    "AllowedValues": [ "10.200.0.0/20" ]  
  }  
}
```

Exemplo de parâmetro

Nome lógico (exclusivo dentro do modelo).

É necessário especificar um tipo de dados compatível (*String*, *Number*, *List*, *CommaDelimitedList*, um parâmetro específico da AWS ou um parâmetro do AWS Systems Manager).

Use a seção opcional **Parâmetros** para personalizar seus modelos. Os parâmetros permitem que você insira valores personalizados no seu modelo toda vez que criar ou atualizar uma pilha.

No exemplo, o parâmetro recebeu o *nome lógico* do *VPCCIDR*. O nome lógico fornecido deve ser exclusivo em todo o modelo.

Os parâmetros do AWS CloudFormation oferecem suporte aos seguintes *tipos de dados*: strings, números, listas delimitadas por vírgulas, tipos de parâmetros específicos da AWS e tipos de parâmetros do AWS Systems Manager (SSM).

Use a seção opcional **Parâmetros** para personalizar seus modelos. Os parâmetros permitem que você insira valores personalizados no seu modelo toda vez que criar ou atualizar uma pilha.

No exemplo, o parâmetro recebeu o *nome lógico* do *VPCCIDR*. O nome lógico fornecido deve ser exclusivo em todo o modelo.

Os parâmetros do AWS CloudFormation oferecem suporte aos seguintes *tipos de dados*: strings, números, listas delimitadas por vírgulas, tipos de parâmetros específicos da AWS e tipos de parâmetros do AWS Systems Manager (SSM).

Os tipos de parâmetros específicos da AWS são particularmente úteis. Por exemplo, quando a pilha for inicializada do console, um parâmetro que você insere como `AWS::EC2::KeyPair::KeyName` renderizará uma lista suspensa que inclui todos os pares de chaves na conta.

Para obter mais detalhes, consulte a página [Parâmetros](#) na documentação do AWS CloudFormation.

Fazer referência a um parâmetro

Para fazer referência a um parâmetro:

- Use a função intrínseca `Ref` para fazer referência a um parâmetro nomeado.
- É convertido em string, independentemente do tipo na declaração de Parâmetro.

Parâmetro

```
"KeyName": {"Type": "AWS::EC2::KeyPair::KeyName"}
```

Faça referência a um parâmetro

```
"KeyName": {"Ref": "KeyName"},
```

- Use a função `Fn::Select` para selecionar valores de uma lista delimitada por vírgulas.

```
"AvailabilityZone" : { "Fn::Select" : [ "0", { "Ref" : "AvailableAZs" } ] }
```

Depois de definir um parâmetro no modelo, você pode fazer referência ao parâmetro usando a função intrínseca `Ref`. Quando você usa a `Ref`, o AWS CloudFormation usa o valor do parâmetro para provisionar a pilha.

Os parâmetros podem ser referenciados nas seções **Recursos** e **Saídas** do mesmo modelo. O exemplo define um parâmetro chamado `KeyName`. Esse parâmetro é referenciado posteriormente, na seção **Recursos** do modelo do AWS CloudFormation. Nesse caso, o tipo de dados do parâmetro é `AWS::EC2::KeyPair::KeyName`. Quando você decidir que deseja criar uma pilha desse modelo, o Console de gerenciamento da AWS renderizará uma lista suspensa com todos os pares de chaves na conta. Você deve selecionar um par

Depois de definir um parâmetro no modelo, você pode fazer referência ao parâmetro usando a função intrínseca *Ref*. Quando você usa a *Ref*, o AWS CloudFormationAWS usa o valor do parâmetro para provisionar a pilha.

Os parâmetros podem ser referenciados nas seções **Recursos** e **Saídas** do mesmo modelo. O exemplo define um parâmetro chamado *KeyPairName*. Esse parâmetro é referenciado posteriormente, na seção **Recursos** do modelo do AWS CloudFormation. Nesse caso, o tipo de dados do parâmetro é *AWS::EC2::KeyPair::KeyName*. Quando você decidir que deseja criar uma pilha desse modelo, o Console de gerenciamento da AWS renderizará uma lista suspensa com todos os pares de chaves na conta. Você deve selecionar um par de chaves na lista antes de criar a pilha.

Às vezes, você pode querer fazer referência a um valor específico em uma lista. Nesses casos, você pode usar a função intrínseca *Fn::Select* na seção **Recursos** do seu modelo. *Fn::Select* retorna um único objeto de uma lista de objetos por índice. No exemplo, o primeiro valor armazenado na lista de *AvailableAZs* será retornado.

Ref e outras funções intrínsecas

A função Ref:

- Permite fazer referência a componentes definidos em um modelo do AWS CloudFormation.
- É necessário para:
 - Referência a parâmetros;
 - Uso de mapas;
 - Junção de strings;
 - Uso de outras funções.

```
"MyEIP" : {  
  "Type" : "AWS::EC2::EIP",  
  "Propriedades" : {  
    "InstanceId" : { "Ref" : "MyEC2Instance" }  
  }  
}
```

Você pode encontrar uma pergunta com frequência ao gravar modelos: como você faz referência a um elemento de um modelo de outro elemento? Por exemplo, como associar instâncias do Amazon EC2 a um endereço IP elástico definido no mesmo modelo?

No exemplo, você pode ver o uso da função intrínseca mais comum, *Ref*, em um fragmento do AWS CloudFormation que define um endereço IP elástico. O uso da função Ref permite que você faça referência a uma instância do Amazon EC2 definida no próprio modelo, que estabelece uma cadeia de relacionamentos entre seus recursos. Um padrão semelhante pode ser usado para associar grupos de segurança com recursos, tabelas de rotas com rotas e instâncias do Amazon

Você pode encontrar uma pergunta com frequência ao gravar modelos: como você faz referência a um elemento de um modelo de outro elemento? Por exemplo, como associar instâncias do Amazon EC2 a um endereço IP elástico definido no mesmo modelo?

No exemplo, você pode ver o uso da função intrínseca mais comum, *Ref*, em um fragmento do AWS CloudFormation que define um endereço IP elástico. O uso da função *Ref* permite que você faça referência a uma instância do Amazon EC2 definida no próprio modelo, que estabelece uma cadeia de relacionamentos entre seus recursos. Um padrão semelhante pode ser usado para associar grupos de segurança com recursos, tabelas de rotas com rotas e instâncias do Amazon EC2 com sub-redes.

Tenha cuidado ao usar a função *Ref*. O valor retornado pela *Ref* depende do recurso fornecido a ela. Por exemplo, usar a função *Ref* em um bucket do Amazon S3 retorna o nome do bucket, mas usá-la em uma fila do Amazon Simple Queue Service (Amazon SQS) retorna o URL da fila. Certifique-se de que o recurso referenciado com a função *Ref* retorna o identificador correto no contexto atual. No exemplo, a função *Ref* retorna o ID de instância de uma

instância do EC2, e é apropriado atribuí-lo à propriedade *InstanceId* do objeto *AWS::EC2::EIP*.

Para obter uma lista dos valores retornados por vários recursos do AWS CloudFormation, consulte a [página Ref](#).

Para selecionar outros atributos em recursos da AWS definidos no modelo do AWS CloudFormation, use a função intrínseca Fn:GetAtt. Para obter mais informações, consulte a [página Fn::GetAtt](#).

Para obter mais detalhes sobre funções intrínsecas em geral, consulte [Referência de função intrínseca](#).

Pseudoparâmetros

Além disso:

- Você pode usar a função intrínseca `Ref` para acessar informações sobre o seu ambiente de tempo de execução.
 - Exemplos: região, nome da pilha, ID da conta da AWS.
- Os pseudoparâmetros são predefinidos: não é necessário especificá-los na seção **Parâmetros** do seu modelo.

```
"Outputs" {  
  "MyStacksRegion" : { "Value" : { "Ref" : "AWS::Region" } }  
}
```

A função intrínseca *Ref* também é usada frequentemente para acessar informações sobre o seu ambiente de tempo de execução, como a região, o nome da pilha ou o ID da conta da AWS. Esses valores de ambiente de tempo de execução estão disponíveis por meio de pseudoparâmetros.

Os *pseudoparâmetros* são parâmetros predefinidos pelo AWS CloudFormation. Você não os declara no seu modelo. Você pode usá-los da mesma forma que usaria qualquer outro parâmetro: como o argumento para a função *Ref*.

No exemplo, *AWS::Region* é um pseudoparâmetro. Ele será resolvido para a região em que o modelo do AWS CloudFormation está sendo executado para criar a pilha.

A função intrínseca *Ref* também é usada frequentemente para acessar informações sobre o seu ambiente de tempo de execução, como a região, o nome da pilha ou o ID da conta da AWS. Esses valores de ambiente de tempo de execução estão disponíveis por meio de pseudoparâmetros.

Os *pseudoparâmetros* são parâmetros predefinidos pelo AWS CloudFormation. Você não os declara no seu modelo. Você pode usá-los da mesma forma que usaria qualquer outro parâmetro: como o argumento para a função *Ref*.

No exemplo, *AWS::Region* é um pseudoparâmetro. Ele será resolvido para a região em que o modelo do AWS CloudFormation está sendo executado para criar a pilha.

Para obter mais informações sobre pseudoparâmetros, consulte [Referências de pseudoparâmetros](#).

Definir mapeamentos em um modelo

Para definir mapeamentos no modelo:

- Defina tabelas de pesquisa de pares de nome-valor em um mapa de dois níveis.
- Combine várias tabelas de mapeamento para criar pesquisas aninhadas.
- É usado com mais frequência para pesquisar os valores atuais da imagem de máquina da Amazon (AMI).

Lembre-se de que uma das seções *opcionais* do modelo é a seção **Mapeamentos**. Um mapeamento corresponde a uma chave a um conjunto correspondente de valores nomeados. Por exemplo, digamos que você deseje definir valores com base em uma região. Você pode criar um mapeamento que use o nome da região como uma chave e que também contenha os valores que você deseja especificar para cada região específica. Você usa a função intrínseca `Fn::FindInMap` para recuperar valores em um mapa.

Os mapeamentos são normalmente usados para se referir aos IDs de AMI das AMIs mais atuais em uma região porque as AMIs diferem por região. Eles também mudam com o tempo à medida que novas AMIs são lançadas.

Você não pode incluir parâmetros, pseudoparâmetros ou funções intrínsecas no

Lembre-se de que uma das seções *opcionais* do modelo é a seção **Mapeamentos**. Um mapeamento corresponde a uma chave a um conjunto correspondente de valores nomeados. Por exemplo, digamos que você deseje definir valores com base em uma região. Você pode criar um mapeamento que use o nome da região como uma chave e que também contenha os valores que você deseja especificar para cada região específica. Você usa a função intrínseca `Fn::FindInMap` para recuperar valores em um mapa.

Os mapeamentos são normalmente usados para se referir aos IDs de AMI das AMIs mais atuais em uma região porque as AMIs diferem por região. Eles também mudam com o tempo à medida que novas AMIs são lançadas.

Você não pode incluir parâmetros, pseudoparâmetros ou funções intrínsecas na seção **Mapeamentos**.

Para obter mais informações, consulte [Mapeamentos](#).